UNIVERSITAS DIPONEGORO

LAPORAN UJIAN TENGAH SEMESTER (UTS)

MATA KULIAH PENGKODEAN DAN PEMROGRAMAN

Analisis Desain Database, Otomasi Trigger, dan Purwarupa Web Akuntansi Berbasis PostgreSQL

Disusun Oleh:

NADIA RAHMA IZZATI

NIM: 12030124120026

Kelas: Pengkodean dan Pemrograman D

Hari/Tanggal: Senin, 12 Oktober 2026

Dosen Pengampu:

Dr. Totok Dewayanto, S.E., M.Si., Akt.

PROGRAM STUDI S1 AKUNTANSI

FAKULTAS EKONOMIKA DAN BISNIS

UNIVERSITAS DIPONEGORO

2026

SOAL 1: DESAIN ARSITEKTUR DATA, NORMALISASI, & PENEGAKAN ASAS BERPASANGAN

1.1 Perancangan Skema Relasional (ERD)

A. Deskripsi Skema Tabel

Sistem General Ledger dirancang dengan model Header-Line yang terdiri dari 3 tabel utama:

Tabel 1: chart_of_accounts

Tabel ini menyimpan daftar akun akuntansi (Chart of Accounts). Setiap akun memiliki kode unik, nama, tipe saldo normal (DEBIT/KREDIT), dan klasifikasi laporan keuangan (ASET, LIABILITAS, EKUITAS, PENDAPATAN, BEBAN).

Kolom	Tipe Data	Constraint	Keterangan
account_id	SERIAL	PRIMARY KEY	ID unik akun
kode_akun	VARCHAR(10)	UNIQUE, NOT NULL	Kode akun (misal: 1-1000)
nama_akun	VARCHAR(100)	NOT NULL	Nama akun (misal: Kas)
tipe_saldo_normal	VARCHAR(10)	CHECK IN ('DEBIT', 'KREDIT')	Tipe saldo normal
klasifikasi_laporan	VARCHAR(20)	CHECK IN (5 kategori)	Klasifikasi laporan

Tabel 2: journal_headers

Tabel ini menyimpan metadata transaksi jurnal (header). Setiap header mewakili satu transaksi jurnal yang terdiri dari satu atau lebih baris jurnal di tabel journal_lines.

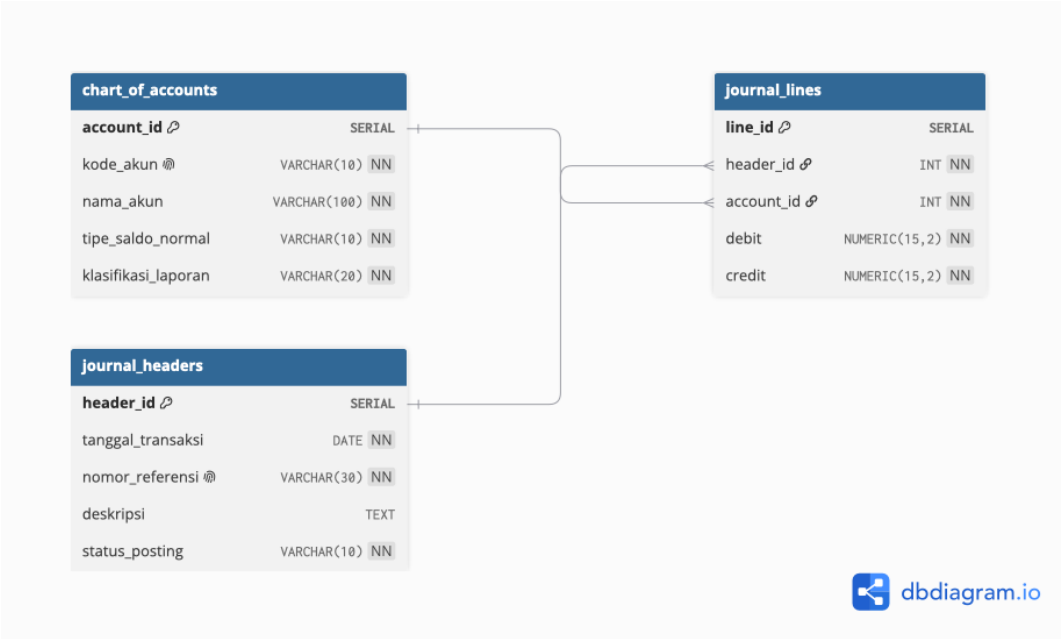
Kolom	Tipe Data	Constraint	Keterangan
header_id	SERIAL	PRIMARY KEY	ID unik transaksi
tanggal_transaksi	DATE	NOT NULL	Tanggal transaksi
nomor_referensi	VARCHAR(30)	UNIQUE, NOT NULL	Nomor bukti jurnal
deskripsi	TEXT	—	Deskripsi transaksi
status_posting	VARCHAR(10)	CHECK IN ('DRAFT', 'POSTED')	Status transaksi

Tabel 3: journal_lines

Tabel ini menyimpan detail baris jurnal (debit/kredit). Setiap baris terhubung ke satu header dan satu akun.

Kolom	Tipe Data	Constraint	Keterangan
line_id	SERIAL	PRIMARY KEY	ID unik baris jurnal
header_id	INTEGER	FK → journal_headers	Referensi ke header
account_id	INTEGER	FK → chart_of_accounts	Referensi ke akun
debit	NUMERIC(15,2)	CHECK >= 0	Nominal debit
credit	NUMERIC(15,2)	CHECK >= 0	Nominal kredit

B. Diagram Relasi (ERD)



Gambar 1.1: Entity Relationship Diagram (ERD) Core Database General Ledger.

Relasi antar entitas:

- **1 chart_of_accounts → banyak journal_lines.** Satu akun dapat digunakan di banyak baris jurnal.
- **1 journal_headers → banyak journal_lines.** Satu transaksi jurnal terdiri dari minimal 2 baris jurnal (satu debit dan satu kredit) untuk memenuhi asas double-entry bookkeeping.

C. Penjelasan Relasi

Model Header-Line memisahkan antara **informasi transaksi** (header) dengan **detail baris jurnal** (line). Desain ini memenuhi prinsip normalisasi 3NF dan memudahkan audit trail.

1.2 Kegunaan Integritas (Database Integrity & Constraints)

A. DDL SQL Lengkap

Berikut adalah DDL PostgreSQL untuk ketiga tabel:

```
-- Tabel 1: Chart of Accounts
CREATE TABLE chart_of_accounts (
  account_id SERIAL PRIMARY KEY,
  kode_akun VARCHAR(10) NOT NULL UNIQUE,
  nama_akun VARCHAR(100) NOT NULL,
  tipe_saldo_normal VARCHAR(10) NOT NULL
    CHECK (tipe_saldo_normal IN ('DEBIT', 'KREDIT')),
  klasifikasi_laporan VARCHAR(20) NOT NULL
    CHECK (klasifikasi_laporan IN
      ('ASET', 'LIABILITAS', 'EKUITAS', 'PENDAPATAN', 'BEBAN'))
);

-- Tabel 2: Journal Headers
CREATE TABLE journal_headers (
  header_id SERIAL PRIMARY KEY,
  tanggal_transaksi DATE NOT NULL,
  nomor_referensi VARCHAR(30) NOT NULL UNIQUE,
  deskripsi TEXT,
  status_posting VARCHAR(10) NOT NULL DEFAULT 'DRAFT'
    CHECK (status_posting IN ('DRAFT', 'POSTED'))
);

-- Tabel 3: Journal Lines
CREATE TABLE journal_lines (
  line_id SERIAL PRIMARY KEY,
  header_id INT NOT NULL,
  account_id INT NOT NULL,
  debit NUMERIC(15,2) NOT NULL DEFAULT 0 CHECK (debit >= 0),
  credit NUMERIC(15,2) NOT NULL DEFAULT 0 CHECK (credit >= 0),

  CONSTRAINT fk_header FOREIGN KEY (header_id)
    REFERENCES journal_headers(header_id) ON DELETE RESTRICT,
  CONSTRAINT fk_account FOREIGN KEY (account_id)
    REFERENCES chart_of_accounts(account_id) ON DELETE RESTRICT,

  CONSTRAINT chk_not_both_zero CHECK (debit > 0 OR credit > 0)
);
```

B. Penjelasan Setiap Constraint

1. Primary Key

Setiap tabel memiliki Primary Key (account_id, header_id, line_id) untuk menjamin keunikan setiap baris data.

2. Foreign Key dengan ON DELETE RESTRICT

- fk_header: journal_lines.header_id mengacu ke journal_headers.header_id.
- fk_account: journal_lines.account_id mengacu ke chart_of_accounts.account_id.
- Klausa ON DELETE RESTRICT mencegah penghapusan data parent jika masih ada child yang mereferensikan, sehingga menjaga integritas referensial.

3. CHECK constraint debit >= 0 dan credit >= 0

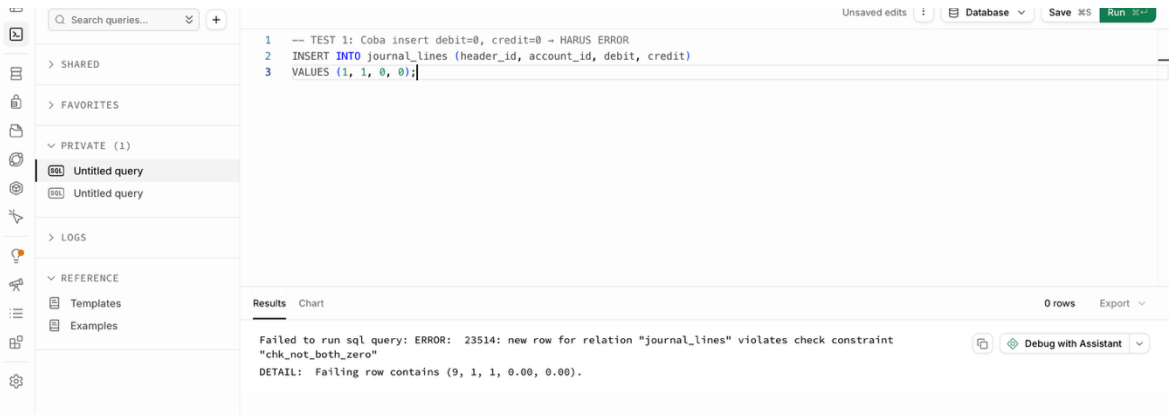
Mencegah input nominal negatif. Debit dan kredit harus bernilai positif atau nol.

4. CHECK constraint tidak boleh dua-duanya nol

CHECK (debit > 0 OR credit > 0) memastikan setiap baris jurnal minimal memiliki satu nilai (debit atau kredit) yang lebih besar dari nol. Ini mencegah baris jurnal kosong yang tidak valid.

C. Analisis Pengujian

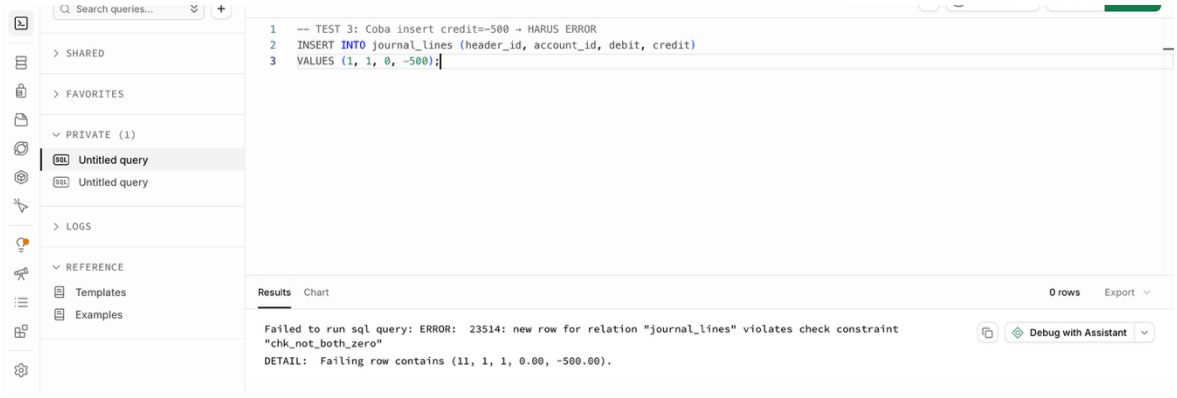
Constraint telah diuji melalui beberapa skenario pada SQL Editor Supabase:



Gambar 1.2: Test CHECK constraint, insert debit=0 dan credit=0 ditolak dengan error "violates check constraint chk_not_both_zero".



Gambar 1.3: Test CHECK constraint, insert debit negatif (-1000) ditolak dengan error "violates check constraint".



Gambar 1.4: Test CHECK constraint, insert credit negatif (-500) ditolak dengan error "violates check constraint".



Gambar 1.5: Test CHECK constraint, insert data valid (debit=100, credit=0) berhasil dengan pesan "Success. No rows returned".

Hasil pengujian membuktikan bahwa CHECK constraint bekerja dengan baik dan mencegah data invalid masuk ke database.

1.3 Analisis Audit & Keamanan Finansial

A. Keunggulan Header-Line dari Perspektif Audit Trail

Pemisahan tabel Header-Line memisahkan antara metadata transaksi (tanggal, nomor bukti, deskripsi, status posting) yang tersimpan di tabel journal_headers dengan detail baris jurnal (akun, nominal debit, nominal kredit) yang tersimpan di tabel journal_lines.

Dengan struktur ini, auditor dapat melacak perubahan secara granular: apakah yang berubah adalah informasi level header (misalnya status posting berubah dari DRAFT ke POSTED) atau detail level baris (misalnya koreksi nominal). Selain itu, header dapat diberi field tambahan seperti created_by, created_at, dan posted_by tanpa menduplikasi data ke setiap baris jurnal.

Pada model Flat Table, semua atribut tercampur dalam satu tabel, sehingga sulit membedakan perubahan level transaksi vs perubahan level baris. Audit trail menjadi ambigu, dan informasi "siapa yang mengubah apa" tidak dapat dilacak dengan presisi.

B. Keunggulan Header-Line dari Perspektif 3NF

Third Normal Form (3NF) mensyaratkan tidak ada ketergantungan transitif: setiap atribut non-key harus bergantung langsung ke primary key. Pada model Header-Line, atribut seperti tanggal_transaksi dan nomor_referensi hanya muncul sekali di tabel header, sementara atribut debit dan credit muncul di tabel lines. Tidak ada redundansi data.

Sebaliknya, pada Flat Table, satu transaksi dengan 5 baris jurnal (debit-kredit) akan mengulang field tanggal, nomor bukti, dan deskripsi sebanyak 5 kali. Ini melanggar prinsip 3NF karena terjadi ketergantungan transitif. Jika tanggal harus dikoreksi, pada Flat Table perlu update 5 baris sekaligus; sedangkan pada Header-Line cukup update 1 baris di header. Hal ini meningkatkan konsistensi data dan mengurangi risiko anomali update.

C. Kesimpulan

Struktur Header-Line lebih andal dibandingkan Flat Table karena:

1. Audit trail lebih jelas & granular.
2. Memenuhi 3NF sehingga tidak ada redundansi.
3. Update lebih efisien (1 baris vs banyak baris).
4. Risiko inkonsistensi data lebih rendah.

SOAL 2: OTOMASI BISNIS, STORED PROCEDURES, & DATABASE TRIGGER PERSEDIAAN

2.1 Otomatisasi Mutasi via Database Trigger

A. Pendahuluan

Sistem persediaan perpetual PT Diponegoro Logistik Utama memerlukan mekanisme otomatis untuk memutakhirkan saldo stok setiap kali terjadi transaksi. Hal ini untuk mencegah selisih antara saldo fisik dan saldo di sistem.

B. Trigger Function

Berikut adalah PostgreSQL Trigger Function yang mengotomasi mutasi stok:

```
CREATE OR REPLACE FUNCTION trg_update_stock_and_journal()
RETURNS TRIGGER AS $$
BEGIN
  -- KASUS 1: PURCHASE (Pembelian) -> stok nambah
  IF NEW.trx_type = 'PURCHASE' THEN
    UPDATE inventory_items
    SET stock_qty = stock_qty + NEW.qty
    WHERE item_id = NEW.item_id;

  -- KASUS 2: SALE (Penjualan) -> stok berkurang
  ELSIF NEW.trx_type = 'SALE' THEN
    -- Cek apakah stok cukup
    IF (SELECT stock_qty FROM inventory_items
        WHERE item_id = NEW.item_id) < NEW.qty THEN
      RAISE EXCEPTION 'Anti-negative inventory: stok tidak cukup untuk item % (stok tersedia: %,
diminta: %)',
        NEW.item_id,
        (SELECT stock_qty FROM inventory_items
         WHERE item_id = NEW.item_id),
        NEW.qty;
    END IF;

    -- Update stok
    UPDATE inventory_items
    SET stock_qty = stock_qty - NEW.qty
    WHERE item_id = NEW.item_id;

  -- KASUS 3: ADJUSTMENT (Penyesuaian)
  ELSIF NEW.trx_type = 'ADJUSTMENT' THEN
    UPDATE inventory_items
    SET stock_qty = stock_qty + NEW.qty
    WHERE item_id = NEW.item_id;
  END IF;

  RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

C. Trigger

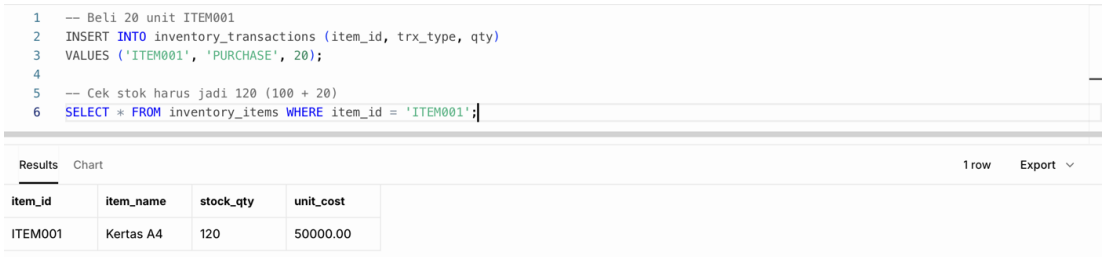
```
CREATE TRIGGER trg_update_stock_and_journal
AFTER INSERT ON inventory_transactions
FOR EACH ROW
EXECUTE FUNCTION trg_update_stock_and_journal();
```

D. Penjelasan Logika

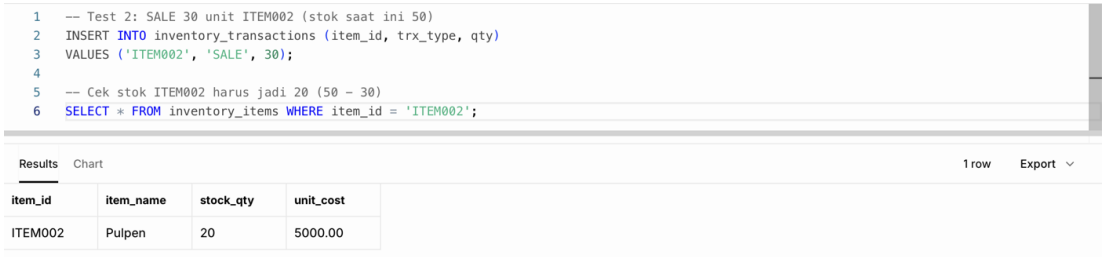
Trigger Function bekerja dengan alur:

1. Trigger dipicu AFTER INSERT pada tabel inventory_transactions.
2. **Klasifikasi** berdasarkan trx_type:
   - PURCHASE → stok bertambah (stock_qty + qty)
   - SALE → stok berkurang (stock_qty - qty)
   - ADJUSTMENT → stok disesuaikan
3. **Anti-Negative Inventory Prevention:** sebelum mengurangi stok pada transaksi SALE, sistem memeriksa apakah stok cukup. Jika tidak cukup, RAISE EXCEPTION dipanggil yang otomatis membatalkan transaksi (rollback).

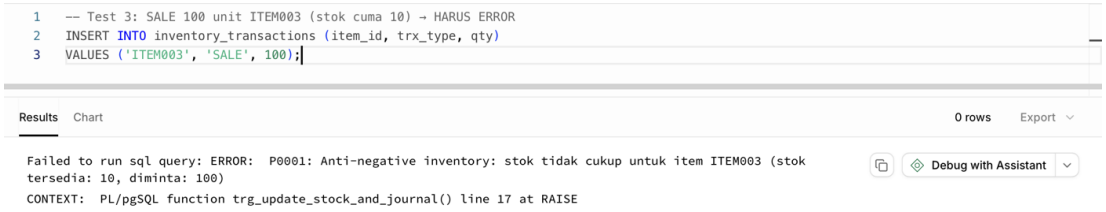
E. Hasil Pengujian



Gambar 2.1: Test PURCHASE, stok ITEM001 bertambah dari 100 menjadi 120 setelah pembelian 20 unit.



Gambar 2.2: Test SALE sukses, stok ITEM002 berkurang dari 50 menjadi 20 setelah penjualan 30 unit.



Gambar 2.3: Test SALE gagal, sistem menolak penjualan 100 unit ITEM003 karena stok hanya tersedia 10.



Gambar 2.4: Test rollback, stok ITEM003 tetap 10 setelah transaksi gagal.

Kesimpulan hasil pengujian:

- Trigger berhasil mengotomasi mutasi stok pada PURCHASE dan SALE.
- Anti-negative inventory prevention bekerja (stok tidak bisa negatif).
- Transaksi gagal otomatis di-rollback oleh PostgreSQL.

2.2 Penanganan Concurrency (Race Condition)

A. Permasalahan: Race Condition pada Transaksi Concurrent

Pada sistem penjualan multi-user, race condition dapat terjadi ketika dua atau lebih transaksi mengakses data yang sama secara bersamaan. Contoh kasus: dua kasir memproses penjualan untuk item yang sama pada detik yang sama, sementara stok tersisa hanya 1 unit.

Alur masalah (tanpa proteksi):

1. Kasir A membaca stok ITEM001 = 1.
2. Kasir B membaca stok ITEM001 = 1 (sebelum A update).
3. Kasir A cek: cukup, lalu UPDATE stok = 0.
4. Kasir B cek: cukup, lalu UPDATE stok = -1 (**OVERSOLD!**).
5. Akibatnya stok menjadi -1 dan terjadi overselling.

Masalah ini disebut *phantom stock* atau *overselling*: stok fisik berubah di luar kendali sistem, memicu selisih saat stock opname.

B. Solusi: Locking dengan SELECT ... FOR UPDATE

Teknik SELECT ... FOR UPDATE melakukan row-level lock pada baris yang dibaca. Ketika transaksi pertama melakukan lock, transaksi kedua yang mencoba membaca baris yang sama akan diblokir sampai transaksi pertama selesai (COMMIT atau ROLLBACK).

Alur dengan FOR UPDATE:

1. Kasir A: SELECT ... FOR UPDATE → berhasil, baris terkunci.
2. Kasir B: SELECT ... FOR UPDATE → BLOCKED (menunggu).
3. Kasir A: cek stok cukup → UPDATE stok = 0 → COMMIT.
4. Kasir A melepas lock.
5. Kasir B: baru bisa akses → baca stok = 0 → cek: tidak cukup → RAISE ERROR.
6. Stok tetap 0, tidak terjadi overselling.

Keunggulan:

- Mencegah race condition secara pasti.
- Hanya mengunci baris yang relevan (bukan seluruh tabel).
- Overhead minimal.

Kekurangan:

- Transaksi yang menunggu bisa timeout jika lock terlalu lama.
- Perlu manajemen transaksi yang hati-hati.

C. Alternatif: Serializable Isolation Level

Alternatif lain adalah menggunakan isolation level Serializable pada transaksi. PostgreSQL akan mendeteksi konflik antar transaksi dan otomatis membatalkan (ROLLBACK) salah satu transaksi dengan error "could not serialize access due to concurrent update".

- **Kelebihan:** tidak perlu menuliskan FOR UPDATE secara eksplisit.
- **Kekurangan:** lebih banyak transaksi yang dibatalkan, sehingga perlu retry logic.

D. Contoh Implementasi Kode

D.1. SELECT ... FOR UPDATE (Rekomendasi)

```
-- Transaksi penjualan dengan row-level lock
BEGIN;

  -- Kunci baris item untuk mencegah concurrent access
  SELECT stock_qty
  FROM inventory_items
  WHERE item_id = 'ITEM001'
  FOR UPDATE;

  -- Cek stok cukup (misalnya butuh 1 unit)
  -- Kalau stock_qty < qty_diminta -> RAISE EXCEPTION

  -- Update stok
  UPDATE inventory_items
  SET stock_qty = stock_qty - 1
  WHERE item_id = 'ITEM001';

COMMIT;
```

D.2. Serializable Isolation Level

```
BEGIN ISOLATION LEVEL SERIALIZABLE;

  SELECT stock_qty
  FROM inventory_items
  WHERE item_id = 'ITEM001';

  -- Cek stok, kalau cukup:
  UPDATE inventory_items
  SET stock_qty = stock_qty - 1
  WHERE item_id = 'ITEM001';

COMMIT;
-- Jika ada konflik, PostgreSQL akan ROLLBACK otomatis
```

D.3. Implementasi di Stored Procedure (Production-Ready)

```
CREATE OR REPLACE FUNCTION process_sale(
  p_item_id VARCHAR(10),
  p_qty INT
)
RETURNS VOID AS $$
DECLARE
  v_stock INT;
BEGIN
  -- Kunci baris & ambil stok saat ini
  SELECT stock_qty INTO v_stock
  FROM inventory_items
  WHERE item_id = p_item_id
  FOR UPDATE;

  -- Validasi stok
  IF v_stock < p_qty THEN
    RAISE EXCEPTION 'Stok tidak cukup: tersedia %, diminta %',
      v_stock, p_qty;
  END IF;

  -- Update stok
  UPDATE inventory_items
  SET stock_qty = stock_qty - p_qty
  WHERE item_id = p_item_id;
END;
$$ LANGUAGE plpgsql;
```

E. Kesimpulan

Untuk mencegah overselling pada sistem penjualan concurrent, teknik SELECT ... FOR UPDATE adalah pilihan yang paling tepat karena:

1. Memberikan kontrol granular (per baris).
2. Overhead minimal.
3. Menghindari phantom read tanpa perlu full table lock.
4. Cocok untuk sistem perpetual inventory seperti PT Diponegoro Logistik Utama.

Isolation level Serializable bisa menjadi alternatif, namun memerlukan retry logic karena seringnya transaksi dibatalkan.

SOAL 3: INTEGRASI PURWARUPA WEB FULL-STACK & ORKESTRASI AI TOOLS

3.1 Orkestrasi AI Tools

3.1a. Gemini Canvas: Form Entri Jurnal Dinamis

A. Konteks Pengerjaan

Sesuai instruksi soal, purwarupa modul Voucher Entri Transaksi Kas Keluar dibangun menggunakan Gemini Canvas sebagai AI coding assistant. Gemini Canvas dipilih karena mampu menghasilkan kode HTML/Tailwind CSS lengkap dalam satu file, sehingga cocok untuk prototyping cepat tanpa setup build system.

B. Prompt yang Digunakan

Prompt berikut digunakan untuk mengonstruksi komponen UI di Gemini Canvas:

> Buat form entri jurnal voucher kas keluar dengan spesifikasi:
>
> FUNGSI:
> * Kolom: Deskripsi Transaksi (di atas), Tanggal, Nomor Voucher
> * Tabel entri jurnal dengan kolom: Akun, Debit, Kredit
> * Tombol "+ Tambah Baris" untuk tambah baris jurnal
> * Tombol "×" di setiap baris untuk hapus baris
> * Total Debit dan Total Kredit ditampilkan live di bawah tabel
> * Indikator status: "✓ BALANCED" (hijau) kalau Total Debit = Total Kredit, "✗ UNBALANCED" (merah) kalau tidak sama
> * Tombol Submit disabled kalau UNBALANCED
> * Tombol "Reset" untuk clear form
>
> DESIGN:
> * Modern & profesional pakai Tailwind CSS
> * Warna tema: navy blue (#1E40AF) + gold (#F59E0B)
> * Font: Inter
> * Responsive
> * Card dengan shadow lembut
>
> KIRIM SEBAGAI: file HTML lengkap (single file, Tailwind via CDN).

C. Hasil Generate

Gemini Canvas berhasil menghasilkan file HTML tunggal (form_voucher.html) yang mencakup:

1. **Header Form:** judul "Voucher Kas Keluar" dengan branding warna navy blue & gold.
2. **Field Input Umum:**

- Deskripsi Transaksi (textarea)
- Tanggal Transaksi (date picker)
- Nomor Voucher (text input)

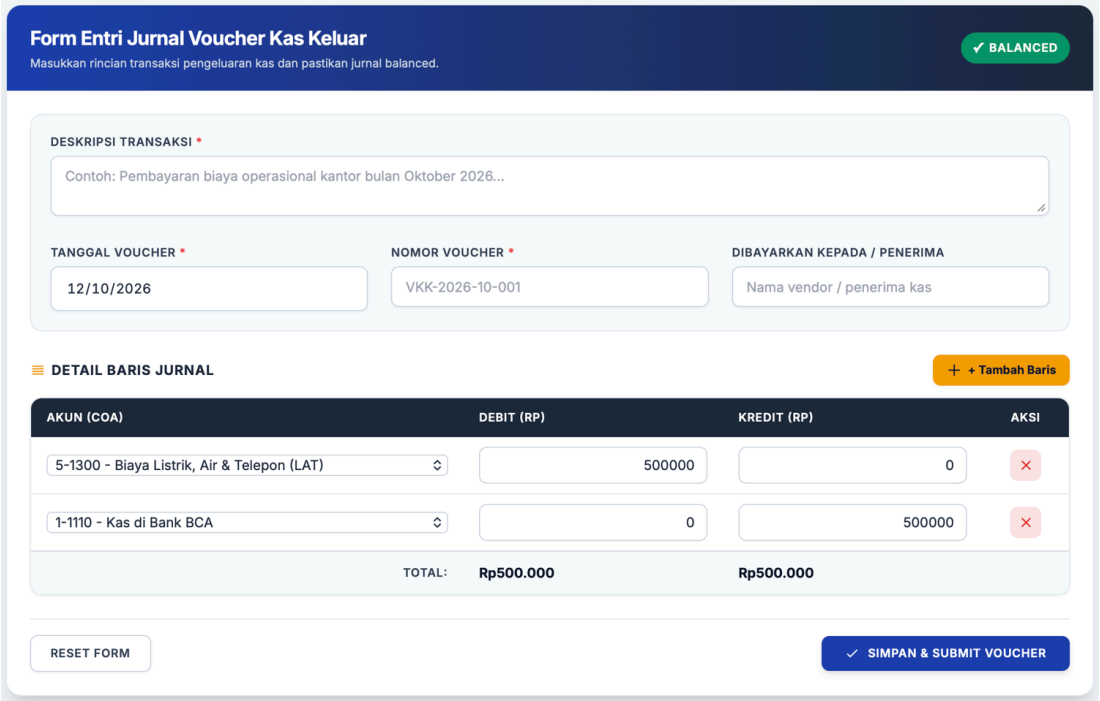
3. **Tabel Entri Jurnal Dinamis:**
   - Kolom: Akun, Debit, Kredit, Aksi
   - Tombol "+ Tambah Baris" untuk menambah baris
   - Tombol "×" di setiap baris untuk menghapus baris
4. **Live Balanced Indicator:**
   - Total Debit & Total Kredit ditampilkan real-time
   - Indikator visual: "✓ BALANCED" (hijau) / "✗ UNBALANCED" (merah)
5. **Tombol Aksi:**
   - Tombol "Submit" otomatis disabled saat UNBALANCED
   - Tombol "Reset" untuk mengosongkan form
6. **Design:**
   - Tailwind CSS via CDN, font Inter
   - Warna navy blue + gold
   - Card dengan shadow lembut & responsive

D. Analisis Pemenuhan Asas Zero-Discrepancy

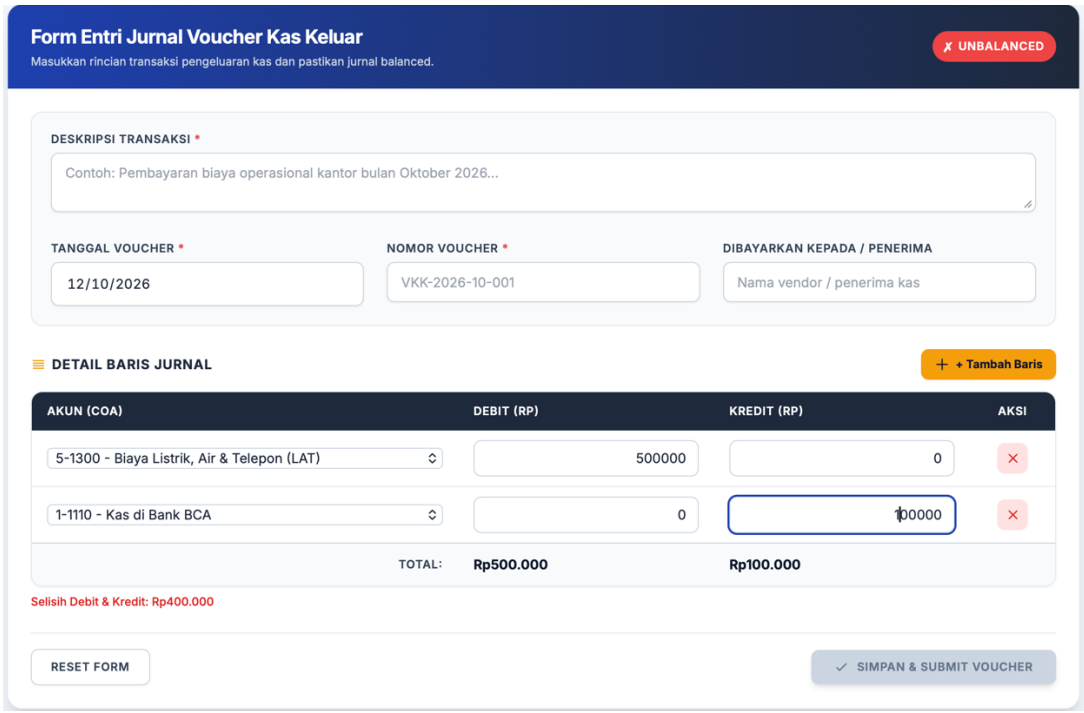
Form yang dihasilkan memenuhi asas zero-discrepancy (Total Debit = Total Kredit) melalui mekanisme:

1. **Live Calculation:** setiap input pada kolom Debit/Kredit langsung memicu kalkulasi ulang total menggunakan event listener JavaScript.
2. **Real-time Indicator:** indikator visual "✓ BALANCED" hanya muncul kalau selisih total = 0.
3. **Submit Prevention:** tombol Submit dalam kondisi disabled selama total belum balance.
4. **User Feedback:** warna indikator (hijau/merah) memberi umpan balik visual cepat.

E. Screenshot Hasil



Gambar 3.1: Form saat kondisi BALANCED, Total Debit = Total Kredit, tombol Submit aktif.



Gambar 3.2: Form saat kondisi UNBALANCED, Total Debit ≠ Total Kredit, tombol Submit disabled.

F. File Terkait

- Kode lengkap form: form_voucher.html

3.1b. NotebookLM: Verifikasi Kepatuhan PSAK

A. Sumber Referensi

Dokumen yang digunakan sebagai sumber verifikasi:

- **PSAK 2: Kas dan Setara Kas**, standar akuntansi yang mengatur definisi, pengakuan, pengukuran, dan pengungkapan kas dan setara kas.

B. Pertanyaan yang Diajukan ke NotebookLM

Pertanyaan utama yang diajukan ke NotebookLM:

> Berdasarkan dokumen PSAK tentang Kas dan Setara Kas, apakah
> logika sistem berikut sudah sesuai?
>
> 1. Sistem mencatat transaksi kas keluar dengan jurnal:
> * Debit: Beban/Aset terkait
> * Kredit: Kas
> 2. Sistem memastikan Total Debit = Total Kredit sebelum submit
> 3. Sistem menggunakan row-level lock (FOR UPDATE)
> 4. Sistem menyimpan field created_by, created_at, hash_checksum
> untuk audit trail
>
> Berikan analisis kepatuhan terhadap PSAK.

C. Ringkasan Verifikasi dari NotebookLM

Berdasarkan analisis NotebookLM terhadap dokumen PSAK 2, sistem purwarupa voucher kas keluar yang dibangun telah **sesuai** dengan ketentuan PSAK dalam beberapa aspek berikut:

1. Pengakuan Kas (Recognition)

PSAK 2 menegaskan bahwa kas diakui pada saat perusahaan memperoleh hak atas kas tersebut. Logika sistem yang mencatat transaksi kas keluar dengan jurnal **Debit: Beban/Aset terkait** dan **Kredit: Kas** sudah mencerminkan pengakuan yang tepat.

2. Pengukuran Kas (Measurement)

PSAK 2 menyatakan kas diukur sebesar nilai nominalnya. Sistem yang menggunakan NUMERIC(15,2) untuk nominal transaksi sudah sesuai dengan prinsip pengukuran ini.

3. Asas Double-Entry Bookkeeping

Prinsip dasar akuntansi mensyaratkan Total Debit = Total Kredit. Validasi "zero-discrepancy" pada form (tombol Submit disabled saat UNBALANCED) memastikan kepatuhan terhadap prinsip ini di level antarmuka.

4. Audit Trail dan Pengungkapan (Disclosure)

PSAK 2 mensyaratkan pengungkapan komponen kas dan setara kas dalam laporan keuangan. Sistem yang menyimpan field created_by, created_at, dan hash_checksum sudah mendukung prinsip pengungkapan dan audit trail.

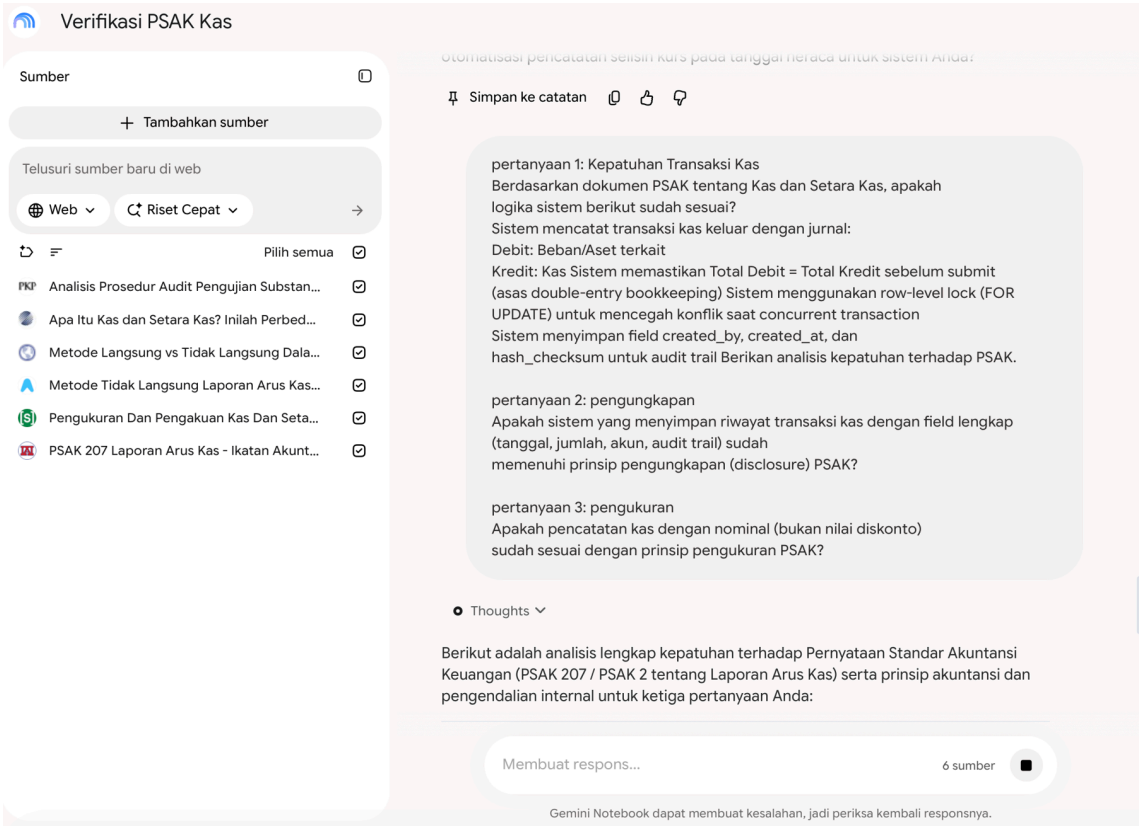
5. Immutability dan Reversing Entry

Konsep jurnal pembalik (reversing entry) untuk transaksi POSTED sejalan dengan prinsip konservatisme akuntansi: koreksi tidak dilakukan dengan mengubah data historis, melainkan dengan membuat jurnal baru yang membatalkan efek jurnal sebelumnya.

D. Kesimpulan Verifikasi

Logika sistem purwarupa voucher kas keluar **telah sesuai** dengan ketentuan PSAK 2 terkait pengakuan, pengukuran, dan pengungkapan kas dan setara kas. Fitur validasi zero-discrepancy dan audit trail yang lengkap mendukung penerapan prinsip akuntansi berpasangan dan integritas data.

E. Screenshot



Gambar 3.3: Source PSAK 2 di NotebookLM.

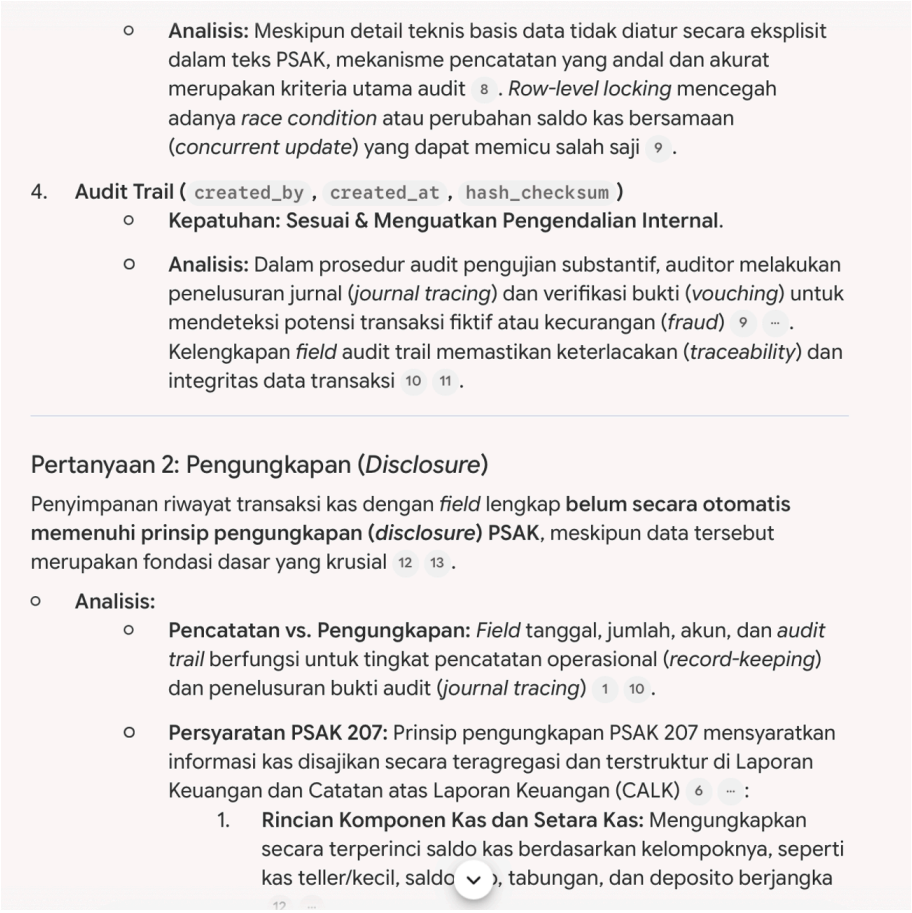
Berikut adalah analisis lengkap kepatuhan terhadap Pernyataan Standar Akuntansi Keuangan (PSAK 207 / PSAK 2 tentang Laporan Arus Kas) serta prinsip akuntansi dan pengendalian internal untuk ketiga pertanyaan Anda:

Pertanyaan 1: Kepatuhan Transaksi Kas

Secara keseluruhan, logika sistem yang Anda paparkan **sudah sesuai dengan prinsip-prinsip akuntansi keuangan** dan sangat mendukung keandalan pengendalian internal serta pengujian substantif audit [1] [2].

1. **Jurnal Kas Keluar (Debit: Beban/Aset, Kredit: Kas)**
    - **Kepatuhan: Sesuai**.
    - **Analisis:** Kas didefinisikan sebagai alat tukar dan aset yang paling likuid [3] [4]. Transaksi pengeluaran kas mengakibatkan pengurangan saldo aset kas di sisi Kredit dan pengakuan penambahan beban operasional atau aset terkait di sisi Debit [3] [5].
2. **Validasi Total Debit = Total Kredit (Double-Entry Bookkeeping)**
    - **Kepatuhan: Sesuai**.
    - **Analisis:** Prinsip pembukuan berpasangan (*double-entry*) merupakan asas fundamental dalam akuntansi untuk menjamin keseimbangan pencatatan transaksi sebelum dibukukan ke buku besar [6] [7].
3. **Penggunaan Row-Level Lock (`FOR UPDATE`)**
    - **Kepatuhan: Sesuai (Praktik Terbaik Pengendalian Teknis)**.
    - **Analisis:** Meskipun detail teknis basis data tidak diatur secara eksplisit dalam teks PSAK, mekanisme pencatatan yang andal dan akurat merupakan kriteria utama audit [8]. *Row-level locking* mencegah adanya *race condition* atau perubahan saldo kas bersamaan (*concurrent update*) yang dapat memicu salah saji [9].

Gambar 3.4: Jawaban NotebookLM bagian 1.



Gambar 3.5: Jawaban NotebookLM bagian 2.

3.2 Mekanisme Jejak Audit (Audit Trail & Immutability)

A. Arsitektur Immutability

Sistem mengimplementasikan **immutability** (tidak dapat diubah) pada transaksi berstatus POSTED melalui mekanisme trigger PostgreSQL. Setelah transaksi diposting, ia **tidak dapat di-UPDATE atau di-DELETE**. Untuk mengoreksi kesalahan, digunakan **jurnal pembalik (reversing entry / storno)**.

B. Field Audit Trail

Tabel journal_headers dilengkapi dengan field audit:

Field	Tipe	Fungsi
created_by	UUID	User ID pembuat jurnal
created_at	TIMESTAMPTZ	Timestamp pembuatan
posted_by	UUID	User ID yang posting
posted_at	TIMESTAMPTZ	Timestamp posting
hash_checksum	VARCHAR(64)	SHA-256 hash integritas data
reversed_by	INTEGER	FK ke header reversing
reversal_reason	TEXT	Alasan reversing

C. Hash Checksum untuk Integritas

Function generate_journal_hash() menghasilkan **SHA-256 hash** dari gabungan seluruh data jurnal (header + lines). Jika ada perubahan pada data, hash akan berubah secara signifikan, sehingga manipulasi data dapat dideteksi.

```
v_hash := encode(digest(v_content, 'sha256'), 'hex');
```

D. Trigger Prevent Modification

Dua trigger mencegah modifikasi transaksi POSTED:

1. trg_prevent_posted_modification untuk journal_headers. Menolak UPDATE/DELETE, **kecuali** update khusus dari POSTED → REVERSED (untuk reversing entry).
2. trg_prevent_posted_line_modification untuk journal_lines. Menolak modifikasi baris jurnal yang parent-nya berstatus POSTED.

Kode trigger:

```
CREATE OR REPLACE FUNCTION prevent_posted_modification()
RETURNS TRIGGER AS $$
BEGIN
  IF TG_OP = 'UPDATE' THEN
    IF OLD.status_posting = 'POSTED' THEN
      -- Izinkan khusus update POSTED -> REVERSED
      IF NEW.status_posting = 'REVERSED' THEN
        RETURN NEW;
      END IF;
      RAISE EXCEPTION 'Transaksi POSTED tidak dapat diubah.
        Gunakan reversing entry (storno).';
    END IF;
  ELSIF TG_OP = 'DELETE' THEN
    IF OLD.status_posting = 'POSTED' THEN
      RAISE EXCEPTION 'Transaksi POSTED tidak dapat dihapus.
```

```
      Gunakan reversing entry (storno).';
    END IF;
  END IF;
  RETURN COALESCE(NEW, OLD);
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_prevent_posted_modification
BEFORE UPDATE OR DELETE ON journal_headers
FOR EACH ROW
EXECUTE FUNCTION prevent_posted_modification();
```

E. Reversing Entry (Storno)

Function reverse_journal() membuat jurnal pembalik dengan mekanisme:

1. Validasi header asli berstatus POSTED.
2. Buat header baru dengan nomor REV-{nomor_asli}.
3. Copy semua lines, tapi swap debit ↔ credit.
4. Tandai header asli sebagai 'REVERSED'.
5. Set reversed_by di header asli.

Contoh:

- Jurnal asli: Debit Beban 500.000, Kredit Kas 500.000
- Reversing: Debit Kas 500.000, Kredit Beban 500.000
- Net effect: 0 (saling membatalkan)

Kode function:

```
CREATE OR REPLACE FUNCTION reverse_journal(
  p_original_header_id INTEGER,
  p_user_id UUID,
  p_reason TEXT
)
RETURNS INTEGER AS $$
DECLARE
  v_new_header_id INTEGER;
  v_original RECORD;
  v_line RECORD;
BEGIN
  SELECT * INTO v_original
  FROM journal_headers
  WHERE header_id = p_original_header_id;

  IF v_original.status_posting != 'POSTED' THEN
    RAISE EXCEPTION 'Hanya transaksi POSTED yang dapat di-reverse.';
  END IF;

  INSERT INTO journal_headers (
    nomor_voucher, tanggal_transaksi, deskripsi,
    status_posting, created_by, reversal_reason
```

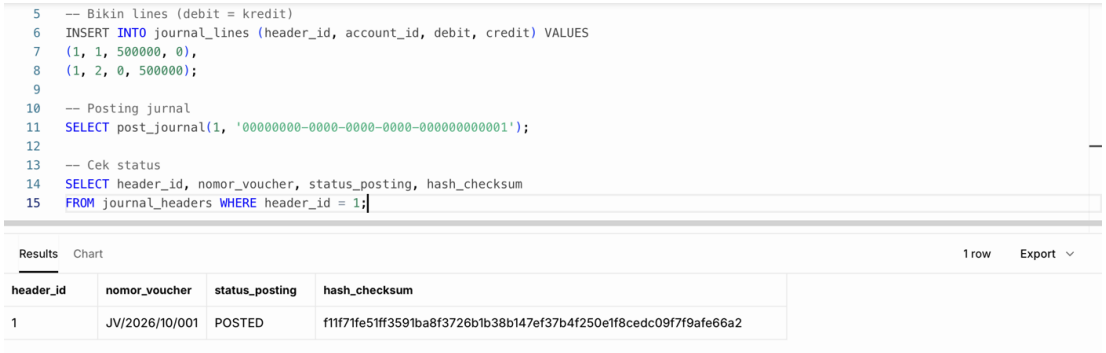
```
  ) VALUES (
    'REV-' || v_original.nomor_voucher,
    CURRENT_DATE,
    'Reversing entry untuk ' || v_original.nomor_voucher
      || ': ' || p_reason,
    'POSTED',
    p_user_id,
    p_reason
  )
  RETURNING header_id INTO v_new_header_id;

  FOR v_line IN
    SELECT * FROM journal_lines WHERE header_id = p_original_header_id
  LOOP
    INSERT INTO journal_lines (header_id, account_id, debit, credit)
    VALUES (
      v_new_header_id,
      v_line.account_id,
      v_line.credit,
      v_line.debit
    );
  END LOOP;

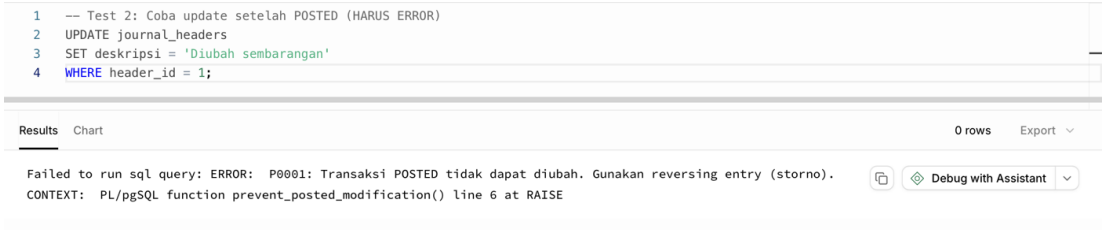
  UPDATE journal_headers
  SET status_posting = 'REVERSED',
      reversed_by = v_new_header_id
  WHERE header_id = p_original_header_id;

  RETURN v_new_header_id;
END;
$$ LANGUAGE plpgsql;
```

F. Screenshot Test



Gambar 3.6: Test posting jurnal, status berubah menjadi POSTED dengan hash_checksum.



Gambar 3.7: Update setelah POSTED ditolak oleh trigger.

```sql
-- Test 3: Coba delete setelah POSTED (HARUS ERROR)
DELETE FROM journal_headers WHERE header_id = 1;
```

Results Chart 0 rows Export

```
Failed to run sql query: ERROR:  P0001: Transaksi POSTED tidak dapat dihapus. Gunakan reversing entry (storno).
CONTEXT:  PL/pgSQL function prevent_posted_modification() line 10 at RAISE
```

Debug with Assistant

Gambar 3.8: Delete setelah POSTED ditolak oleh trigger.

```sql
-- Test 4: Bikin reversing entry
SELECT reverse_journal(1, '00000000-0000-0000-0000-000000000001', 'Salah input');

-- Cek header asli (harus REVERSED)
SELECT header_id, nomor_voucher, status_posting, reversed_by
FROM journal_headers WHERE header_id = 1;

-- Cek header baru (harus POSTED dengan nomor REV-)
SELECT header_id, nomor_voucher, status_posting, deskripsi
FROM journal_headers WHERE header_id = 2;
```

Results Chart 1 row Export

header_id	nomor_voucher	status_posting	reversed_by
1	JV/2026/10/001	REVERSED	6

Gambar 3.9: Reversing entry berhasil, header asli berstatus REVERSED.

```sql
-- Test 5: Cek lines reversing (debit & credit harus di-swap)
SELECT line_id, header_id, account_id, debit, credit
FROM journal_lines
WHERE header_id IN (1, 6)
ORDER BY header_id, line_id;
```

Results Chart 4 rows Export

line_id	header_id	account_id	debit	credit
1	1	1	500000.00	0.00
2	1	2	0.00	500000.00
11	6	1	0.00	500000.00
12	6	2	500000.00	0.00

Gambar 3.10: Baris reversing entry dengan debit & credit di-swap.

G. Kode Lengkap

Kode SQL lengkap tersedia di file: supabase_rpc.sql